Νευροασαφής Έλεγχος

Αναφορά 1ης Άσκησης

2023-2024

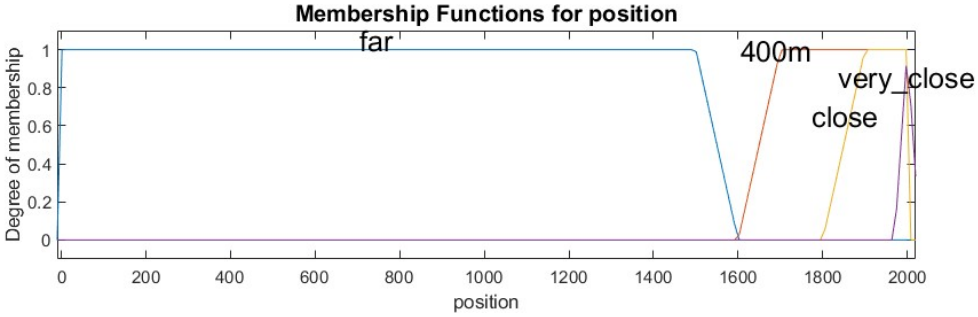
Κόνιας Οδυσσέας Γεώργιος 03119166

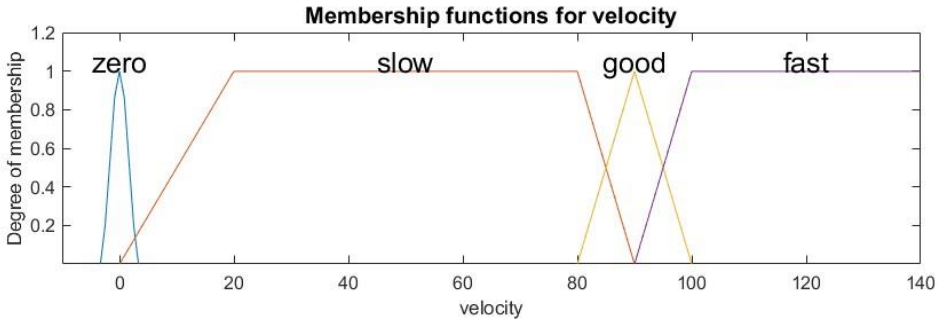
Θέμα 1

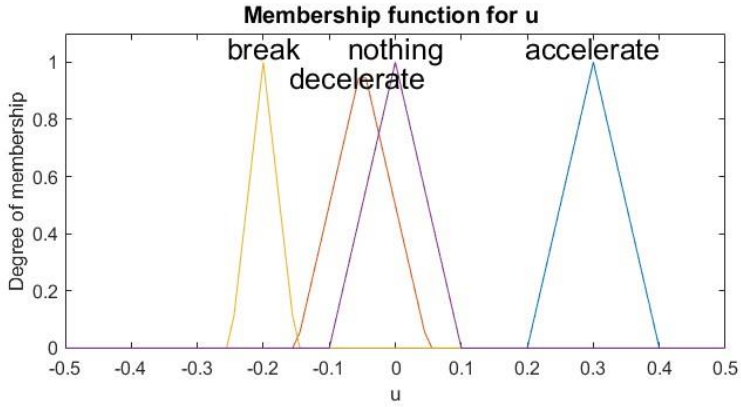
Ο κώδικας για το fuzzy inference system:

```
a1 = 1;
a2 = 0.05;
a3 = 0.1;
b = 200;
m = 100;
F = 0;

v_ref = 90;     %desired velocity
d = 2000;       %train station distance
a = v_ref/90;
%The train fuzzy inference systemm
train = mamfis('Name', 'train');

% Add input variables
train = addInput(train, [-10 d+20], 'Name', 'position');
train = addMF(train, 'position', 'trapmf', [-10 0 d-500 d-400], 'Name', 'far');
train = addMF(train, 'position', 'trapmf', [d-400 d-300 d d], 'Name', '400m');
train = addMF(train, 'position', 'trapmf', [d-200 d-100 d d], 'Name', 'close');
train = addMF(train, 'position', 'trimf', [d-30 d d+30], 'Name', 'very_close');

train = addInput(train, [-10 v_ref+50], 'Name', 'velocity');
train = addMF(train, 'velocity', 'trapmf', [-3 -0.5 0.5 3], 'Name', 'zero');
train = addMF(train, 'velocity', 'trapmf', [0 20 v_ref-10 v_ref], 'Name', 'slow');
train = addMF(train, 'velocity', 'trimf', [v_ref-10 v_ref v_ref+10], 'Name', 'good');
train = addMF(train, 'velocity', 'trapmf', [v_ref v_ref+10 v_ref+50 v_ref+50], 'Name', 'fast');

deceleration = -0.05*a;
breaking = -0.2*a;

% Add output variable: u
train = addOutput(train, [-1 1], 'Name', 'u');
train = addMF(train, 'u', 'trimf', [0.2 0.3 0.4], 'Name', 'accelerate');
train = addMF(train, 'u', 'trimf', [deceleration-0.1 deceleration deceleration+0.1], 'Name',
'decelerate');
train = addMF(train, 'u', 'trimf', [breaking-0.05 breaking breaking+0.05], 'Name', 'break');
train = addMF(train, 'u', 'trimf', [-0.1 0 0.1], 'Name', 'nothing');

rules = [...
    "If (position is far) and (velocity is zero) then u is accelerate";...
    "If (position is far) and (velocity is slow) then u is accelerate";...
    "If (position is far) and (velocity is fast) then u is decelerate";...
    "If position is 400m then u is nothing";...
    "If (position is close) and (velocity is not zero) then u is decelerate";...
    "If (position is very_close) and (velocity is not zero) then u is break"...
];
% Add rules to the FIS
train = addRule(train, rules);
```

Ορίζουμε έναν ασαφή ελεγκτή με εισόδους την θέση και την ταχύτητα του τραίνου και έξοδο το u. Έχουμε τις παρακάτω συναρτήσεις συμμετοχής για την θέση την ταχύτητα και το u:



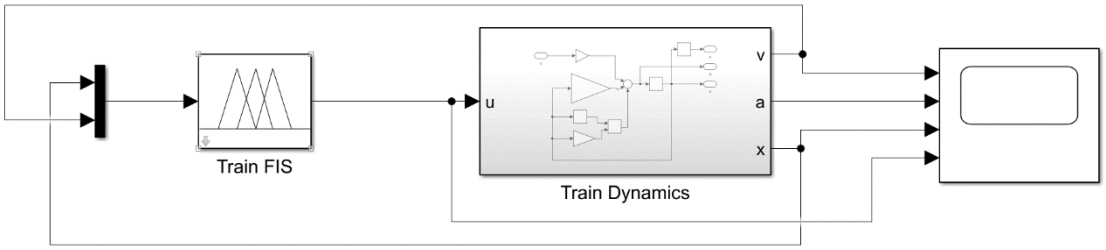




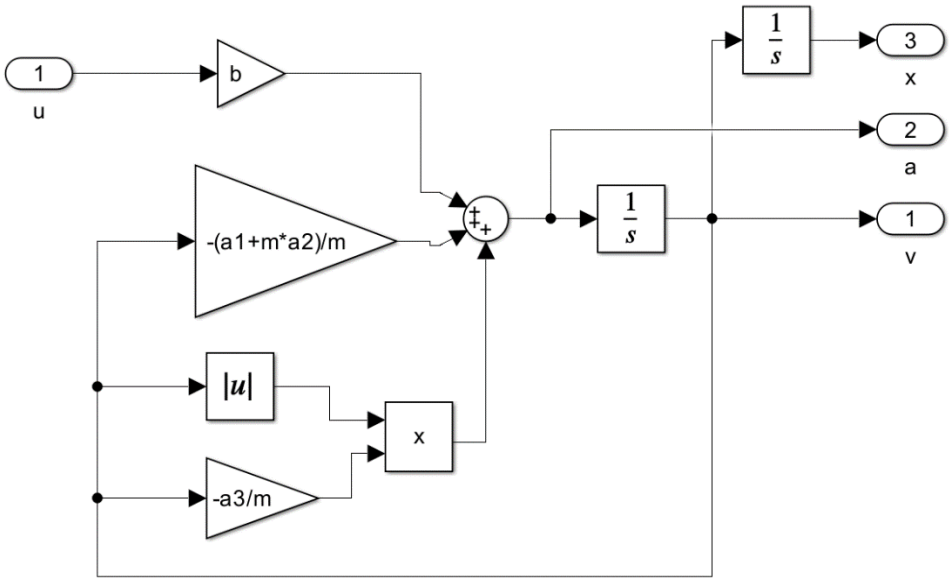
Το τραίνο αρχικά επιταχύνει μέχρι να φτάσει την επιθυμητή ταχύτητα και την διατηρεί. Μόλις πλησιάζει τον σταθμό (πχ 400m) αφήνουμε το τραίνο ελεύθερο ώστε να επιβραδύνει από μόνο του. Όταν είμαστε πιο κοντά στον σταθμό (close) επιβραδύνουμε λίγο και όταν το τραίνο φτάσει πολύ κοντά στον σταθμό επιβραδύνουμε πολύ. Με την λογική αυτή ορίζονται οι κανόνες:

```
"If (position is far) and (velocity is zero) then u is accelerate"
"If (position is far) and (velocity is slow) then u is accelerate"
"If (position is far) and (velocity is fast) then u is decelerate"
"If position is 400m then u is nothing"
"If (position is close) and (velocity is not zero) then u is decelerate"
"If (position is very_close) and (velocity is not zero) then u is break"
```

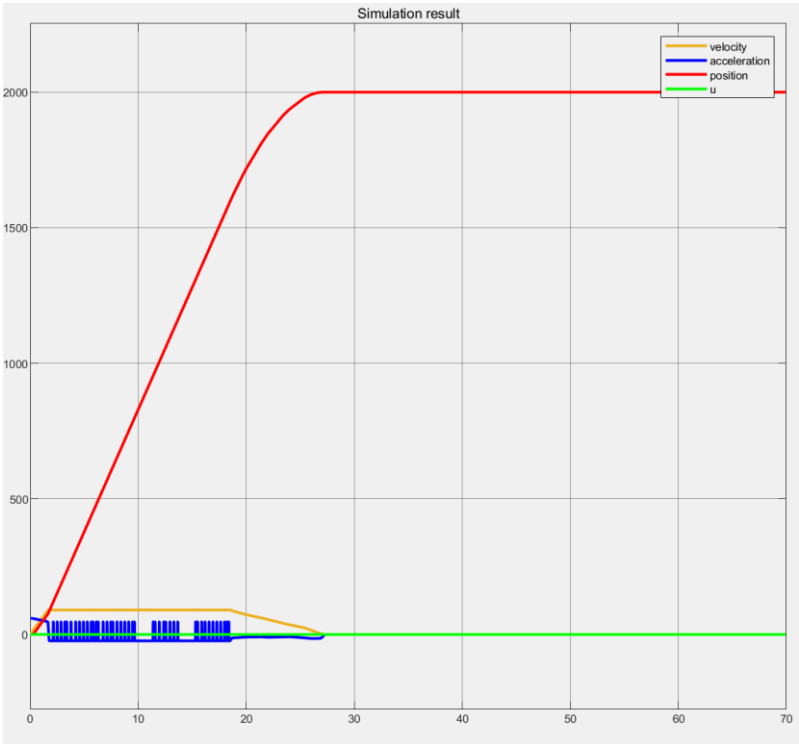
Κάνουμε προσομοίωση στο Simulink για να δοκιμάσουμε τον ελεγκτή:
Το block train dynamics προσομοιώνει την διαφορική εξίσωση του τραίνου. Ο fuzzy ελεγκτής παίρνει ως είσοδο το διάνυσμα του x και του v



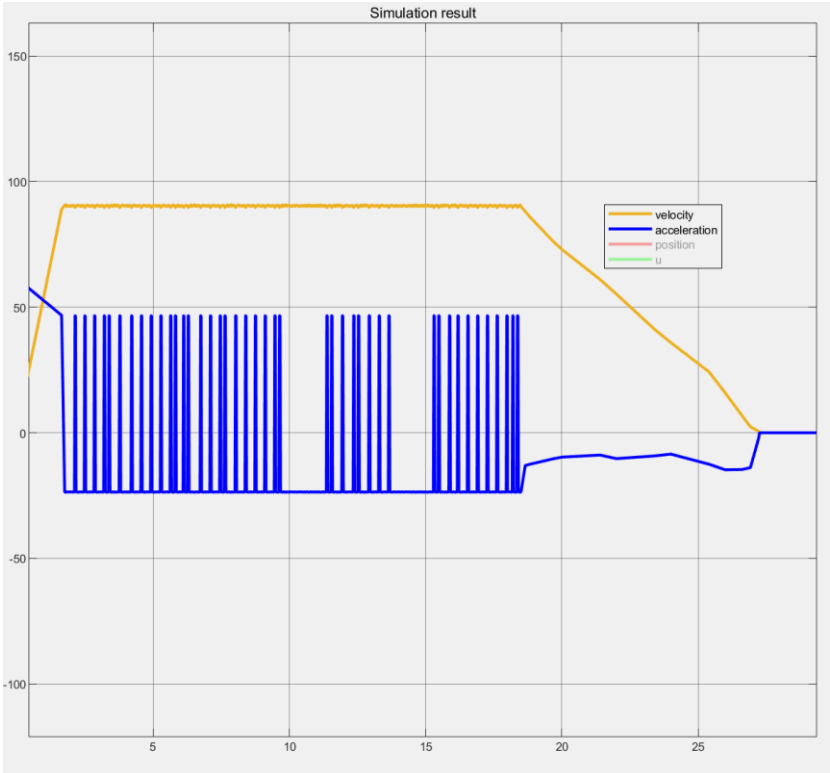
Train Dynamics:



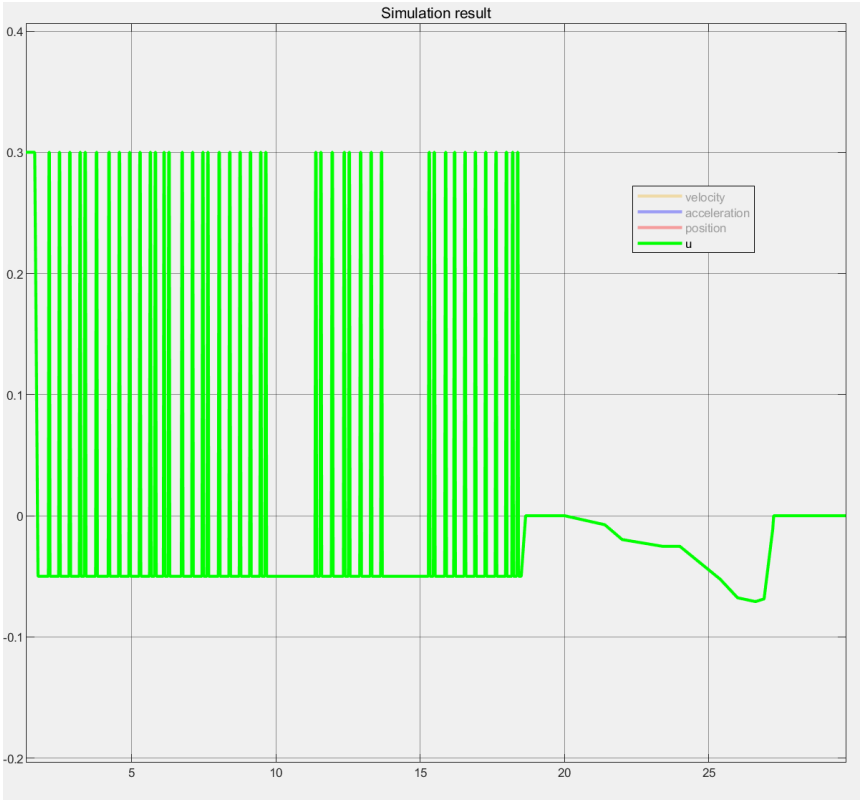
Για ταχύτητα αναφοράς 90m/s και απόσταση του σταθμού 2000m έχουμε το παρακάτω αποτέλεσμα:



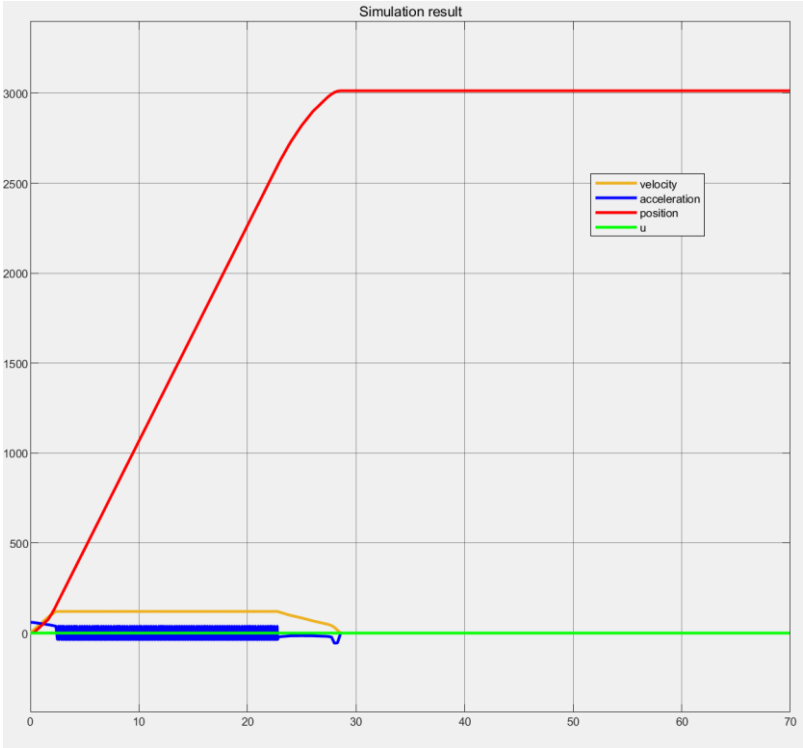
Πετυχαίνουμε ακριβώς την επιθυμητή απόσταση (1999,98m) και ταχύτητα. Βλέπουμε ότι το τραίνο διατηρεί την επιθυμητή ταχύτητα μέχρι που επιβραδύνει με σταθερό σχετικά ρυθμό:



Το σήμα ελέγχου:

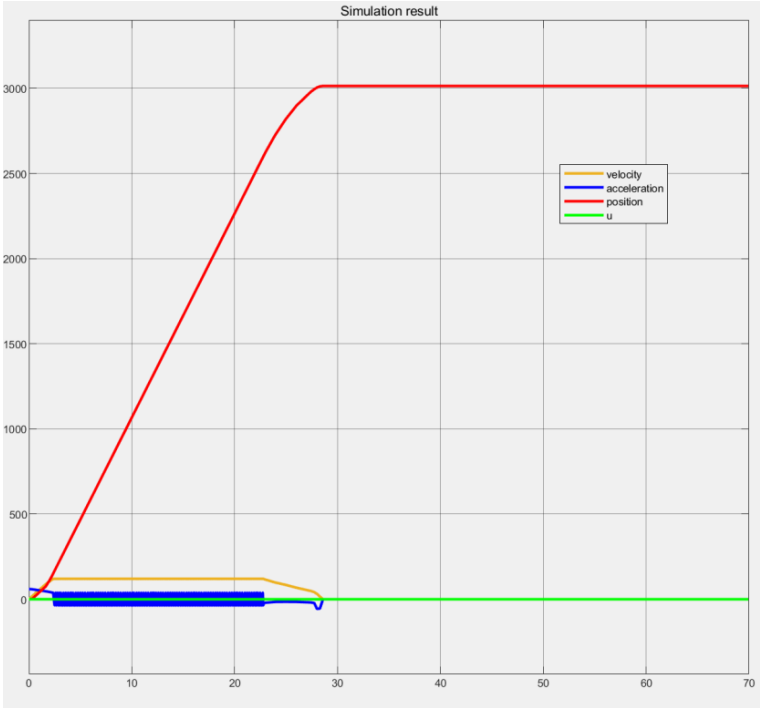


Αποτέλεσμα για επιθυμητή ταχύτητα 120m/s και απόσταση 3000m:



Το τραίνο και πάλι έχει την επιθυμητή ταχύτητα και σταματάει σχεδόν στην επιθυμητή απόσταση (3013m)

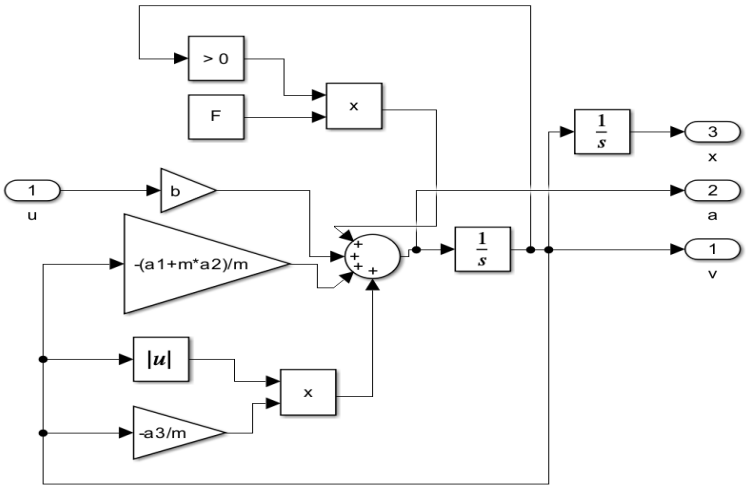
Αποτέλεσμα με επιθυμητη ταχύτητα 90m/s απόσταση 2000m και αύξηση του βάρους κατά 10% m=110:



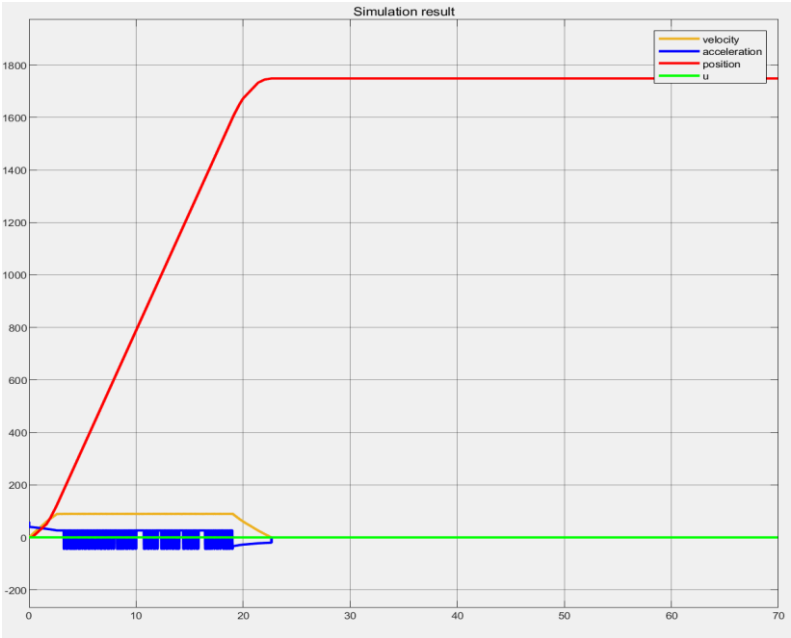
Πετυχαίνουμε το επιθυμητό αποτέλεσμα (2001m)

Δοκιμάζουμε εάν ο έλεγχος λειτουργεί προσθέτοντας εξωτερική δύναμη στην δυναμική του τραίνου. Θεωρούμε ότι η δύναμη είναι αντίθετη στην κατεύθυνση του τραίνου και ασκείται όσο το τραίνο κινείται.

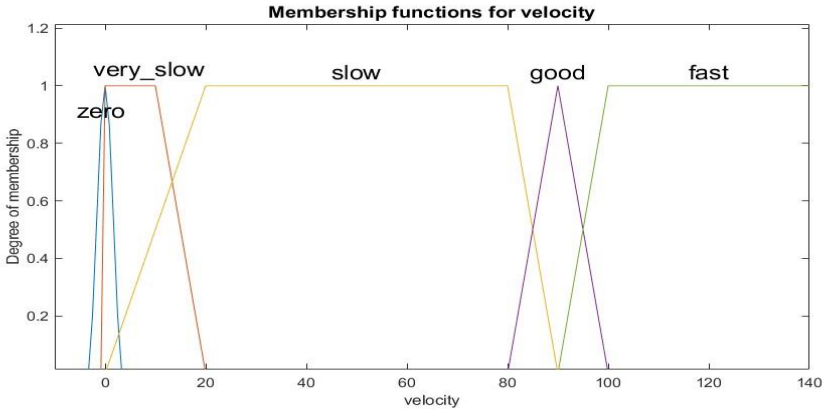
Την προσθέτουμε στο train dynamics ως εξής:



Θέτουμε F=-20 και δοκιμάζουμε:



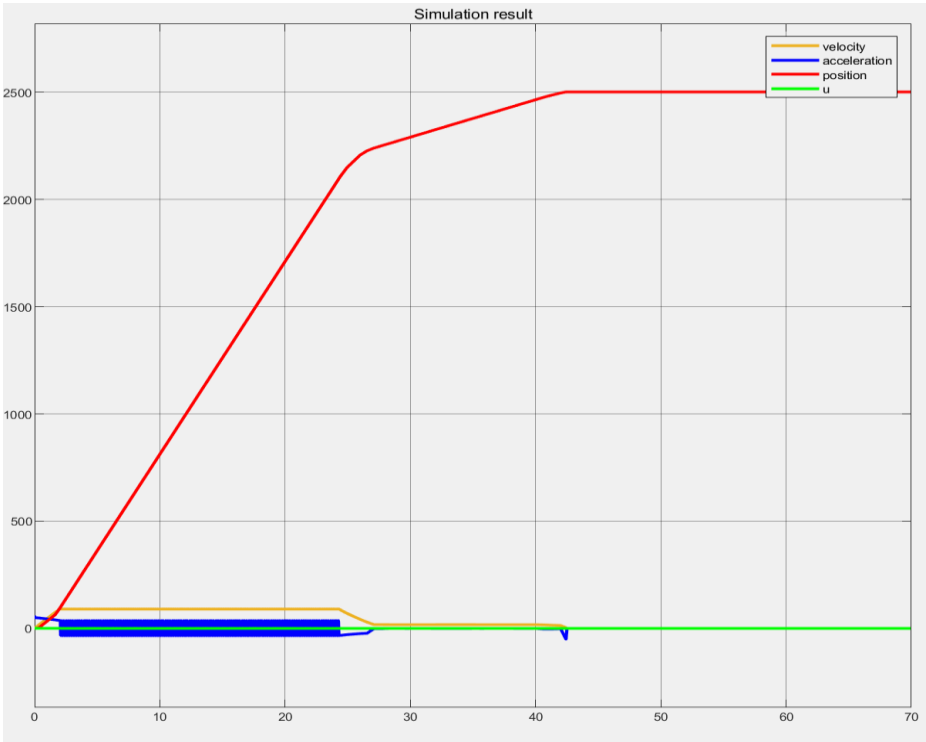
Το τραίνο φτάνει την επιθυμητή ταχύτητα αλλά σταματάει πολύ νωρίς. Χρειάζεται να αλλάξουμε τους κανόνες καθώς αν αφήσουμε το τραίνο να φρενάρει μόνο του θα σταματήσει. Ορίζουμε μια νέα συνάρτηση συμμετοχής για την ταχύτητα (very slow): **`train = addMF(train,'velocity','trapmf',[0 0 10 20],'Name','very_slow');`**



Στόχος είναι το φρενάρισμα να ξεκινάει με το τραίνο να έχει ταχύτητα περίπου 15m/s μέχρι να φτάσει που κοντά στον σταθμό οπού θα φρενάρει πολύ. Οι ανανεωμένοι κανόνες είναι:

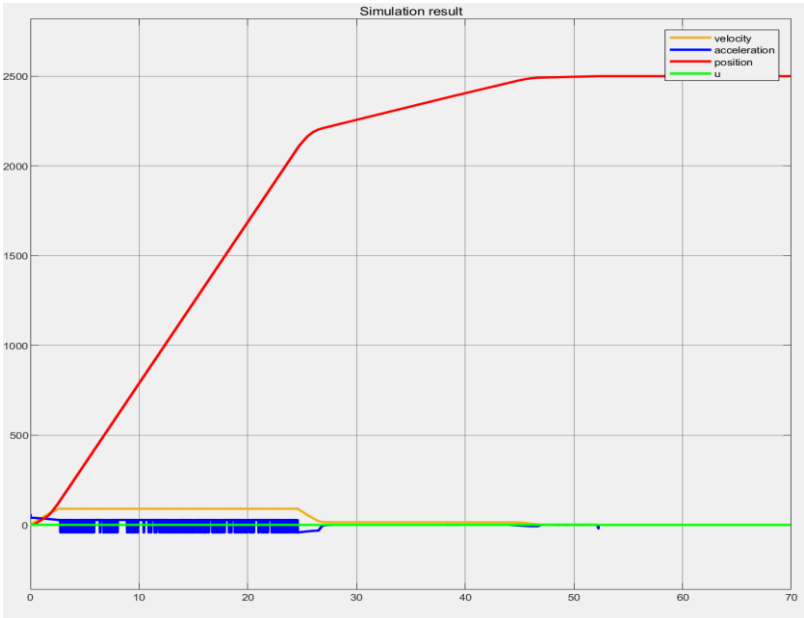
```
"If (position is far) and (velocity is zero) then u is accelerate"
"If (position is far) and (velocity is slow) then u is accelerate"
"If (velocity is fast) then u is decelerate"
"If position is 400m and (velocity is slow) then u is decelerate"
"If position is 400m and (velocity is very_slow) then u is accelerate"
"If (position is close) and (velocity is not zero) then u is decelerate"
"If (position is very_close) and (velocity is not zero) then u is break"
```

Τελικά:



Η επιβράδυνση είναι λιγότερο ομαλή από το αρχικό μοντέλο αλλά πετυχαίνουμε τις προδιαγραφές.

Δοκιμή για v_ref = 120 d = 2500 F= -20 και m =110:



Θέμα 2 Α.

Θα χρησιμοποιήσουμε το cvx για να δούμε αν υπάρχει κατάλληλη συνάρτηση Lyapunov. Θα πρέπει να βρούμε πίνακα P ο οποίος να ικανοποιεί την παρακάτω σχέση:

$A^T P + PA < O,\ \ P > 0 \Rightarrow\ A^T P + PA < -I,\ \ P > I$

Στο cvx το πρόβλημα ορίζεται ως εξής:

```
a = -0.5;
A1 = [0.9 a; 0 0.8];
A2 = A1';
n = length(A1);
cvx_begin sdp quiet
variable P(n,n) symmetric
A1'*P*A1-P<= eye(n)
A2'*P*A2-P<= eye(n)
P >= eye(n)
cvx_end
disp(cvx_status)
disp(P)
```

Τρέχουμε το script για διάφορες τιμές του α:

a = 0 Solved P = 17.0879 -0.0000 -0.0000 16.7350	**a = -0.1** Solved P = 16.4974 -0.0571 -0.0571 16.3832
a = 0.1 Solved P = 16.4974 0.0571 0.0571 16.3832	**a = -0.3** Solved P = 10.8922 1.6474 1.6474 11.9904
a = 0.3 Solved P = 10.8922 -1.6474 -1.6474 11.9904	**a = -0.5** Solved P = 3.7736 1.6524 1.6524 4.4345
a = 0.5 Solved P = 3.7736 -1.6524 -1.6524 4.4345	**a = -1** Infeasible
a = 1 Infeasible	**a = -2** Infeasible
a = 3 Infeasible	**a = -3** Infeasible
a = 5 Infeasible	**a = -5** Infeasible

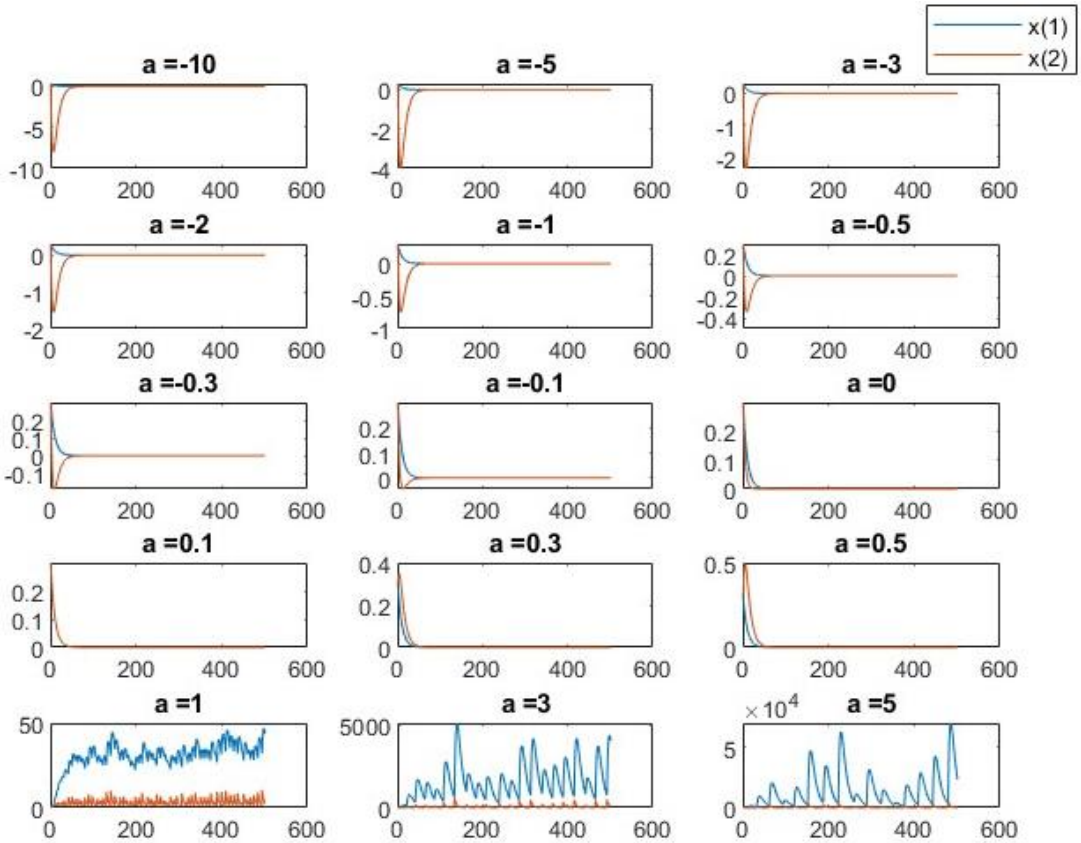
Για τις τιμές στις οποίες βρίσκουμε πίνακα P μπορούμε να αποδείξουμε την ευστάθεια του συνολικού συστήματος με την συνάρτηση: $V = x^T P x$

B.

Υπολογίζουμε την απόκριση του συστήματος με τον παρακάτω κώδικα:

```
i = 0;
values = [-5 -3 -2 -1 -0.5 -0.3 -0.1 0 0.1 0.3 0.5 1 3 5];
for a = values
    A1 = [0.9 a;0 0.8];
    A2 = [0.9 0;a 0.8];
    N = 500;
    x =zeros(2,N);
    x(:,1) = [0.3;0.3];
    for n = 1:N
        h1 = min(max(x(2,n)-0.5,0),1);
        h2 = 1 - h1;
        x(:,n+1) = h1*A1*x(:,n)+h2*A2*x(:,n);
    end
    i = i +1;
    subplot(5, 3, i)
    plot(x(1,:))
    hold on
    plot(x(2,:))
    title(['a =' num2str(a)]);
end
```

Κάνουμε plot την απόκριση του συστήματος για διάφορες τιμές του α για να δούμε εάν τα x(1), x(2) τείνουν στο 0. (Αρχικές συνθήκες (0.3, 0.3))



Το σύστημα φαίνεται να είναι ασυμπτωτικά ευσταθές για τις τιμές που βρήκαμε συνάρτηση Lyapunov αλλά και για άλλες όπως α = -1, -2, -3, -5. Βέβαια παρατηρήσαμε ότι με άλλες αρχικές συνθήκες η απόκριση για κάποια α σταματάει να τείνει στο 0.